



King's College London

SatNet: Skeletal Attention Network

Submitted by:

Mert Arcan
23120474

Project Report Submitted in Partial Fulfillment of the
Requirements for the M.Sc. Project Module

Engineering Department

Supervised by:

Dr. Hak-Keung Lam

London

August / 2024

Abstract

Controlling articulated robots is crucial in various industries that require automation, such as manufacturing, healthcare, and logistics. Traditional control policies are usually based on Multi-Layer Perceptrons (MLPs) and they often struggle in complex environments in context of efficiency and performance. This research introduces the Skeletal Attention Network (SatNet) which is designed to improve the control mechanism of articulated robots, aiming to improve the sample efficiency, precision, decision-making capability and robustness in continuous control tasks. SatNet integrates Graph Attention Networks and Beta distribution into the reinforcement learning framework in order to capture the structure of robot and perform accurate decisions. The performance of SatNet was evaluated with series of experiments in simulated environments where it was compared against traditional MLP policy, and a variant of itself which uses Gaussian Distribution. Results demonstrated that SatNet-Beta significantly improves sample efficiency and robustness in environments where actions follow symmetric patterns, while SatNet-Gaussian excels in more dynamic tasks where exploration is essential. These findings suggest that the integration of GATs with the proper distribution method into control policies have the potential to advance the performance of robotic systems, making them more reliable in various applications.

Keywords: Articulated Robot, Graph Attention Network, Action Selection, Reinforcement Learning, Policy Network, Continuous Control.

Table of Contents

	Page
Abstract	ii
List of Figures	v
List of Tables	vi
List of Acronyms and Notation	vii
Acknowledgements	ix
Chapter 1: Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Proposed Model	2
1.4 Solution Approach, Exciting Outcomes, and Contribution	3
1.5 Outline	3
Chapter 2: Literature Review	4
2.1 Introduction	4
2.2 Related Work	4
2.2.1 Reinforcement Learning and Policy Optimization	4
2.2.2 Graph Neural Networks	5
2.3 Graph Neural Network - Reinforcement Learning Models in Robot Control	6
2.4 Research Gap	7
Chapter 3: Problem Statement	9
3.1 Problem Description	9
3.2 Main Objective	9
3.2.1 Traditional Methods	10
3.2.2 SatNet	11
Chapter 4: Methodology	13
4.1 Introduction	13
4.2 Model Methodology	13
4.2.1 Graph Construction	13
4.2.2 Policy-Value Network Architecture	15

4.2.3	Input Model	16
4.2.4	Propagation Model	16
4.2.5	Output Model	17
4.2.6	Learning Algorithm	18
Chapter 5:	Experiments	19
5.1	Introduction	19
5.2	Experimental Setup and Design	19
5.2.1	Experimental Setup and Computational Environment	19
5.2.2	Experimental Design	19
5.2.3	Constraints, Assumptions, and Model Simplifications	20
5.3	Results	21
5.3.1	Performance Benchmark	21
5.3.2	Robustness of Policies	24
5.4	Inferences	26
5.4.1	Discussion	26
Chapter 6:	Professional and Ethical Issues	27
6.1	Introduction	27
6.2	PESTEL Analysis	27
6.3	Ethical Considerations	29
6.4	Inclusive Engineering Outcomes	29
6.5	Conclusion	29
Chapter 7:	Conclusion	30
References		31
Appendix A:	Raw Data	34
A1	Computer Code	34
A2	Details of the Experimental Settings	47
A2.1	Hyperparameter Selection	47
A2.2	Graph Simplification	48

List of Figures

Figure 3.1.	Network architecture of SatNet, illustrating the flow of data through the policy and value networks. The diagram highlights the use of GAT layers for message aggregation and the integration of Beta distribution for action selection within the policy network. The Alpha and Beta networks, essential for generating the parameters of the Beta distribution, are also depicted, showcasing the structure and functionality of SatNet in controlling articulated robots.	12
Figure 4.1.	The graph structure of the Ant agent. The diagram illustrates the connection of nodes representing the robot’s joints, where the torso serves as the root node, and each hip is connected to its corresponding ankle. .	14
Figure 5.1.	Training performance of SatNet-Beta, SatNet-Gaussian, and MLP over 1 million timesteps in the Ant-v4 , HalfCheetah-v4 , Humanoid-v4 , and HumanoidStandup-v4 environments.	21
Figure 5.2.	Heatmap illustrating the mean reward loss due to noise across different environments for SatNet-Beta, SatNet-Gaussian, and MLP.	24
Figure 5.3.	Violin plots showing the reward distribution of SatNet-Beta, SatNet-Gaussian, and MLP in original and noisy environments for Ant-v4 , Humanoid-v4 , HumanoidStandup-v4 , and HalfCheetah-v4	25
Figure A.1.	Graph Simplification of the Humanoid Structure: The Torso, Abdomen, and Pelvis are Combined into a Single Root Node Labeled as Torso . . .	48

List of Tables

Table 2.1.	Summary of research gaps identified in related work on reinforcement learning, GNNs, and their applications in continuous control tasks. The table provides insights into how these studies inform the design and implementation of SatNet as a control policy for articulated robots. . .	8
Table 5.1.	Comparison of mean reward and standard deviation during testing for SatNet-Beta, SatNet-Gaussian, and MLP across different environments.	23
Table 6.1.	PESTEL Analysis	28
Table A.1.	MLP: Hyperparameter Grid Search Settings	47
Table A.2.	SatNet: Hyperparameter Grid Search Settings	47
Table A.3.	SatNet: Chosen Hyperparameters	48
Table A.4.	MLP: Chosen Hyperparameters	48
Table A.5.	Mean Reward Achieved by MLP, SatNet-GAUSSIAN, and SatNet-BETA Models in Ant-v4 and Humanoid-v4 Environments at 51.200-Timestep Intervals	50
Table A.6.	Mean Reward Achieved by MLP, SatNet-GAUSSIAN, and SatNet-BETA Models in HumanoidStandup-v4 and HalfCheetah-v4 Environments at 51.200-Timestep Intervals	51
Table A.7.	Evaluation Results for Different Models and Environments: Maximum, Mean, and Standard Deviation of Rewards Across Original and Noisy Conditions	52

List of Acronyms and Notation

Acronyms

NN	Neural Network
GNN	Graph Neural Network
RL	Reinforcement Learning
DRL	Deep Reinforcement Learning
MuJoCo	Multi-Joint dynamics with Contact
GAT	Graph Attention Network
WAGNN	Weighted Aggregation Neural Network
TRPO	Trust Region Policy Optimization
PPO	Proximal Policy Optimization
DDPG	Deep Deterministic Policy Gradient
CPU	Central Processing Unit
MLP	Multi Layer Perceptron

Notation

G	Graph
V	Set of nodes in G
E	Set of edges in G
T	Transpose operation
$\mathbf{W}$	Weight matrix
$\parallel$	Concatenation operation
α_{ij}	Attention coefficient
$\mathbf{a}$	Parameter vector for attention
$\mathbf{h}_i$	Hidden representation of node i
$\mathcal{N}_i$	Set of neighbors of node i
h	Range of the action space
Σ	Summation symbol
f	Probability density function
$\pi_\theta(a s)$	Action probability in state s
$\alpha(s, \theta), \beta(s, \theta)$	Parameters of the Beta distribution
$\pi_\theta(a^\tau s^\tau)$	Stochastic policy at time τ
$r(s^\tau, a^\tau)$	Reward at time τ
$P(s^{\tau+1} s^\tau)$	Transition probability
γ	Discount factor
$J(\theta)$	Objective function
$\mathbb{E}_\pi$	Expected value under policy π

Acknowledgements

I wish to extend my thanks to Dr. Hak-Keung Lam for his invaluable support and guidance throughout the development of this research.

Chapter 1

Introduction

1.1 Background

The rapid advancements in technology have led to significant advancements in artificial intelligence and robotics. Recently, numerous amount of research has been focused on reinforcement learning (RL) and artificial intelligence (AI). The huge success of experiments where agents outperform humans at simpler games like Go [1] and more complex ones like Starcraft II [2] has been one of the main forces behind the curiosity in reinforcement learning. The outstanding performance of these agents has accelerated advancements in both RL and robotics. Currently, the field of robotics is still advancing rapidly, driven by the increasing demand for automation in industries such as manufacturing, healthcare, and logistics.

Articulated robots plays a crucial role in these applications due to their ability to perform complex tasks since they have multiple joints and segments which allow them to operate human-level operations. RL has emerged as a promising method for controlling these robots autonomously by learning to adapt, optimize, and enhance their performance across various domains [3]. However, current control policies struggle in complex and dynamic environments. This inconsistency underscores a critical need for innovative approaches to enhance the control policy of articulated robots, bridging the gap between their potential and current control capabilities.

1.2 Problem Statement

Articulated robots are vital in various industries, especially in the ones that require automation, such as manufacturing, healthcare, and logistics. Traditional control policies, such as MLP [4] may work well for simple and well-defined objectives, but they struggle in more complex environments [5]. These environments require enhanced efficiency and robustness which are vital features that traditional controllers often lack [6]. Current approaches to integrate RL with Graph Neural Network (GNN) [7], including NerveNet [8] which demonstrates the potential of GNN-RL integration by performing slightly better than traditional continuous control policies. However,

it can still be enhanced with superior variants of GNN’s and different action selection techniques [9, 10]. The effective control of articulated robots is significant in such environments as it impacts the robot’s ability to perform tasks autonomously and efficiently.

Industries which heavily rely on automation require precise, robust and cost-efficient control policies. Errors in such systems may potentially lead to security issues, far beyond than just time and financial losses, demanding the need for optimized control policies [11]. A novel approach to enhance the model’s policy by leveraging graph structure promises cost reduction, enhanced safety, and increased reliability as the policy network becomes more efficient and robust [8].

Despite the considerable advances made in RL and GNNs, the current integration of these frameworks still suffer from sample and learning inefficiency [12, 13]. Therefore, there is still a significant gap in understanding how to effectively integrate these technologies for controlling articulated robots. This gap limits their effective use in sectors requiring automation since real-world applications become prohibitively expensive and time-consuming due to the extensive training data and computational resources needed. It is essential to fill this knowledge gap to enable more efficient and cost-effective deployment of autonomous systems. By addressing these unresolved issues, this project seeks to develop a more robust and efficient policy in order to improve the autonomy and intelligence of articulated robots. This advancement is crucial for making robotic systems more accessible to use in industries where they are required.

1.3 Proposed Model

This research proposes a novel approach to integrate cutting-edge graph-based and sample distribution methodologies with RL, introducing the Skeletal Attention Network (SatNet). SatNet isolates itself from the state-of-the-art control policies by integrating Graph Attention Network (GAT) [14] and Beta sampling for action selection into its infrastructure rather than using traditional methods like GNN’s and Gaussian distribution. The model will be designed for use in the Gymnasium simulated environments introduced by OpenAI [15] powered by MuJoCo [16] to accomplish continuous control tasks. Therefore, the results may not directly reflect real-world applications.

1.4 Solution Approach, Exciting Outcomes, and Contribution

To further enhance the potential of GNN-RL integration, this research leverages GATs to not only capture the complex morphological structure of an articulated robot but also integrates attention mechanism which significantly improves the model by capturing the structure of the agent’s graph depending on attention and proposes better navigation of continuous control tasks. Beta sampling improves the stochastic policy gradient in continuous control tasks as shown in the paper [10] which claims that ”The Beta policy is bias-free and provides significantly faster convergence and higher scores over the Gaussian sampling”. The benchmark of SatNet, and it’s variant which is detailed in Chapter 5, with MLP policy illustrates that SatNet is more sample efficient than the traditional policies. However, the training process requires exponentially more time compared to traditional methods due to complex operations introduced by GAT layers. While powerful Graphics Processing Units (GPUs) could mitigate this issue, the current research is constrained by hardware limitations.

This research does not use any existing data, instead it generates its own data by training agents in the Gymnasium’s simulated environments for long period of times. The code required for this research was written in Python3 since Python offers extensive libraries that support this research.

This paper claims the SatNet is more sample-efficient and more robust than conventional control policies. These findings have profound implications for the field of robotics and RL, as they pave the way for more efficient and effective training methodologies, advancing the capabilities of autonomous robotic systems and their applications in real-world scenarios.

1.5 Outline

This paper is organized as follows. Chapter 2 reviews the literature on RL, GNNs, including GATs, and their applications in continuous control tasks. Chapter 3 defines the problem and extends the traditional methods and proposes SatNet. Chapter 4 details the methodology. Chapter 5 discusses the results. Chapter 6 discusses the engineering issues resulting from this research. Finally, Chapter 7 summarizes the research and provides potential future extensions.

Chapter 2

Literature Review

2.1 Introduction

This review focuses on how GAT, RL, and cutting-edge methodologies can be integrated to form a precise, robust, cost and computation-efficient controller. Section 2.2 surveys the related work in Reinforcement Learning, Graph Neural Network and its variants whereas Section 2.4 highlights the contributions.

2.2 Related Work

2.2.1 Reinforcement Learning and Policy Optimization

Reinforcement Learning is a type of machine learning where an agent learns to take best decisions by performing actions in an environment and maximizing their cumulative rewards [17]. In continuous control tasks, which are used in this research, RL is useful as it can handle a range of possible actions and adapt to dynamic environments [18]. Policy optimization, a key aspect of RL, involves finding the best policy that dictates the agent’s actions based on it’s current state.

Proximal Policy Optimization (PPO) [19], is a cutting-edge policy gradient method that alternates between sampling data through interaction with the environment and optimizing an objective function using stochastic policy gradient ascent [20]. The papers [5, 21] presents a comprehensive benchmarking of Deep Reinforcement Learning (DRL) algorithms such as Trust Region Policy Optimization (TRPO) [22], Truncated Natural Policy Gradient (TNPG) [23], Reward-Weighted Regression (RWR) [24], and Deep Deterministic Policy Gradient (DDPG) [25] in various environments. Authors claim that the importance of selecting the environment specific algorithm is paramount as there is no single algorithm which outperforms others in every single task.

PPO excels from traditional policy gradient methods by using a clipped surrogate objective which avoids extremely large policy updates that can possibly destabilize learning. It has been claimed that PPO outperforms other policy gradient methods on various benchmark tasks, including simulated robotic locomotion and Atari game playing [19]. Given its outstanding performance,

PPO is a promising candidate for an articulated robot controller by creating a structured policy learning when it is integrated with GATs. In this research all experiments is conducted using PPO as policy optimization method.

In reinforcement learning, Gaussian distribution is often used for selectin the action in continuous control tasks. However, this distribution can potentially introduce bias which is problematic when the actions are bounded by physical constraints, leading to inefficiency learning and lower performance as stated in the paper [10]. This paper analyses the bias and variance of policy gradients under both Gaussian and Beta distributions. Authors claim that their proposed Beta policy, which uses Beta distribution is bias free and offers faster convergence and higher cumulative reward scores compared to Gaussian-based policy when applied to stochastic policy optimisation methods like TRPO and PPO in continuous control problems. SatNet integrates Beta distribution to it's action selection mechanism to eliminate biases and to form a more precise and robust controller.

2.2.2 Graph Neural Networks

There have been many efforts to generalize Neural Networks (NNs) to graph-structured data [26], such as using hash functions to apply Concurrent Neural Networks (CNNs) to graphs. Another approach is founded on Recurrent Neural Networks (RNNs). However, they are limited with specific graph structures [8, 7].

Graph Neural Networks [7] represent a versatile class of models for general graphs. It extends the concept of traditional NNs to graph-structured data, where the data is represented as nodes, edges or both. GNNs use message passing between nodes to capture the dependencies within the graph data. According to the paper [27], GNNs had significant performance improvements in various tasks such as node classification, physical systems modeling and object detection by leveraging the relationships between nodes in the graph.

Building on the foundations of GNNs, this paper [14] presents a significant enhancement. This study introduces Graph Attention Networks (GATs) which is a novel neural network that utilizes masked self-attentional layers to operate graph-structured data. The graph attention layer utilized within these networks does not require costly operations so it is cost-efficient. The key

innovation of GATs lies in their ability to assign dynamic weights to node neighbors, improving the model’s capacity to capture relevant features. The GAT architecture performs state-of-the-art performance across various benchmark datasets. Integrating GATs, which is a variant of GNN, to create GAT-RL controller for articulated robots aligns with this research’s objective of enhancing decision-making and precision by enabling the policy network to focus on the most concerned aspects of robot’s structure. SatNet Utilizes GAT layers by integrating them into their policy network.

2.3 Graph Neural Network - Reinforcement Learning Models in Robot Control

In recent advancements in the field of reinforcement learning and robotics, the ”NerveNet” [8] framework has emerged as a significant contribution, especially for structured policy learning for continuous control tasks. NerveNet models the agent’s morphology as a graph, and feeds it into the GNNs which represents the agent’s policy network, departing from traditional approaches such as using MLPs. The experimental results claims that NerveNet outperforms traditional control policies and excels in structure learning tasks like size and disability transfer. These findings highlight NerveNet’s robustness and generalizability. This study is crucial as it provides insights into how GNN-RL integration can enhance the control of articulated robots, laying a foundation for this research.

The paper [28] presents Weighted Aggregation Graph Neural Network (WAGNN) which addresses the challenges of skill transfer and learning in articulated robots. Major contributions of the paper involves a weighted aggregation strategy to improve policy learning. While traditional GNN models treat all nodes equally during aggregation, WAGNN integrates kinematic analysis, assigning higher weights to the joint nodes that are closer to the center of the robot, to improve both learning speed and control accuracy. The experimental results show that WAGNN surpasses NerveNet model in both zero-shot and few-shot transfer learning scenarios. Therefore, using an attention mechanism in a graph-structured articulated robot can enhances learning speed and control accuracy in continuous control tasks which aligns with the aim of the proposed SatNet.

The paper ”Learning Individual Features to Decompose State Space for Robotic Skill Learn-

ing” [29] tackles robotic skill learning in high-dimensional state spaces by leveraging GNNs to create separate controllers for each joint. This approach reduces the number of parameters in the policy network by using low-dimensional features specific to each joint, leading to improved learning efficiency and reduced overfitting. The study emphasizes the importance of feature decomposition for enhancing the scalability and robustness of GNN-based policies. Inspires consideration of feature decomposition but diverges from unified policy network approach.

The paper [9] introduces DMAP, an attention-based policy network architecture which is inspired by biological systems. The DMAP model integrates independent proprioceptive processing, a distributed policy with distinct controllers for each joint, and an attention mechanism that dynamically directs sensory information from various body parts to corresponding controllers. The authors compare the performance of DMAP with a baseline model with only proprioceptive state information and an oracle model with access to the actual body parameters. Remarkably, DMAP performs better than the oracle model across multiple environments and demonstrates robustness and adaptability. The principles of attention used in DMAP inspires this work for integrating GATs for enhanced performance in robot control.

2.4 Research Gap

Table 2.1 lists studies that investigated factors which contribute to form SatNet as the control policy of articulated robots.

Table 2.1: Summary of research gaps identified in related work on reinforcement learning, GNNs, and their applications in continuous control tasks. The table provides insights into how these studies inform the design and implementation of SatNet as a control policy for articulated robots.

Author	Focus	Methodology	Key Findings	Limitations	Relation to This Research
Schulman et al. (2017) [19]	Proximal Policy Optimization (PPO)	Policy Gradient Methods	PPO outperforms traditional methods	Limited to standard environments	Selecting PPO as RL algorithm
Wang et al. (2018) [8]	NerveNet framework	GNN for policy network	NerveNet outperforms traditional methods	-	Leveraging Graph-based models in policy architecture
Zhang et al. (2020) [28]	WAGNN	Weighted Aggregation in GNN	Improved learning speed and control accuracy	Limited to specific skill learning tasks	Aggregation Strategy
Velickovic et al. (2018) [14]	Graph Attention Networks (GATs)	Attention mechanism in GNN	State-of-the-art performance on benchmark datasets	Focus on node classification	Integration of GATs in RL for robot control
Po-Wai et al. (2017) [10]	Beta Distribution in RL	Beta vs Gaussian distribution	Beta distribution offers bias-free and efficient learning	Limited to standard stochastic policy methods	Application of Beta distribution in action selection
Chiappa et al. [9]	DMAP	Biologically-inspired attention-based policy	Outperforms traditional models in adaptability	Focus on specific locomotion tasks	Leveraging attention-based policy
Zhou et al. (2020) [27]	GNN methods and applications	Review of GNN applications	Significant improvements in various tasks	General without specific application	Significance improvement in physical system modelling with Graph-based methods
Zhang et al. (2020) [29]	Learning Individual Features	GNN-based decomposition with separate controllers	Reduces parameters and overfitting; enhances learning efficiency	Limited handling of dynamic tasks	Informs feature decomposition but diverges from unified policy approach

Chapter 3

Problem Statement

3.1 Problem Description

Controlling articulated robots, which is a continuous control task, presents several technical challenges, including sample inefficiency [25], low precision and weak robustness [30] because of their complex structures with multiple joints, and the multi-dimensional state and action spaces.

Current approaches [8, 9, 31], demonstrate the enormous potential of graph-based policy networks by outperforming traditional continuous control policies, inspiring this research to push the boundaries of robotic control by advancing the current methodologies with comprehensive methods [32]. This research proposes SatNet which integrates GAT layers to enhance decision-making and sample efficiency, and leverages Beta distribution for reliable and precise action selection in reinforcement learning tasks.

The significance of this research promises overall performance enhancement and paves the way for new applications in automation where robots can efficiently learn and make accurate decisions. This research sets its foundation on the recent work done by [8, 14, 10, 33] and aims to integrate these methods in order to bridge the identified gaps in the literature and advance the field of robotic control.

3.2 Main Objective

To start with, understanding the concept and framework of Reinforcement Learning (RL) is crucial to better understand this research. Central to RL is the Markov Decision Process (MDP). MDP's are a class of stochastic sequential decision processes in which the cost and transition functions depend only on the current state and action of the system [34]. In other words, the past observations are not necessary for ideal control. MDP is defined by the tuple (S, A, P, R, γ) where S and A represents the set of states and actions, respectively, while $P(s'|s, a)$ is the transition probability of moving from state s to state s' given action a . $R(s, a)$ is the reward function, providing the reward received after taking action a in state s . Finally, γ is the discount

factor which determines the importance of future rewards.

$$J(\theta) = \mathbb{E}_{\pi} \left[\sum_{\tau=0}^{\infty} \gamma^{\tau} r(s_{\tau}, a_{\tau}) \right]. \quad (3.1)$$

In RL problems, the main aim is to maximize the objective function 3.1 which is the total reward $J(\theta)$, by interacting with the environment.

At each time step τ , the agent receives the observation vector for the current state s_{τ} , selects an action a_{τ} according to its policy $\pi(a|s)$, receives a reward r_{τ} for taking action a_{τ} on state s_{τ} , and transitions to the next state $s_{\tau+1}$ according to the transition probability $P(s_{\tau+1}|s_{\tau}, a_{\tau})$. The main target of the agent is to maximize its cumulative reward..

For instance, the Ant-v4 environment, as described in [35], features a robotic ant composed of a torso with four legs, each leg made up of two segments, as illustrated in Figure 4.1. The objective is to coordinate the movement of all four legs by applying torques to the eight hinges in the intended direction. The applied torques are determined through reinforcement learning, the agent interacts with the environment, processes the observation features and predict how much torque it should apply to all its hinges. Depending on the reward received, the reinforcement learning algorithm attempts to maximize the objective function 3.1 by taking various actions and observing the resulting changes in the environment, iteratively updating its policy until it converges on an optimal or satisfactory policy.

3.2.1 Traditional Methods

Traditional methods for continuous control in RL typically employ Multi-Layer Perceptron (MLP) [4] policies. MLP Policy uses fully connected neural networks to represent the policy and value functions. MLP Policy shows certain levels of success, especially the time required for each iteration is much lower than more complex methods. However, it faces challenges as it is prone to overfitting and limited generalization [31]. Moreover, it struggles when capturing complex dependencies in the articulated robot’s morphology, especially with the ones having a huge number of joints and high dimensional state spaces.

Some of the recent studies [8, 28, 31] structure the articulated robot as a graph and feed it into their GNN-based policy to enhance the overall performance. Yet, they still face challenges

in efficiently handling high-dimensional action spaces as they struggle with the sample-efficiency and robustness.

3.2.2 SatNet

The SatNet’s major components are GAT layers for aggregated message propagation, and Beta distribution for action selection to mitigate the unintended bias issue. The feature extraction process represents the agent as a graph, which is then fed into the policy and value networks, as depicted in Figure 3.1.

The **policy network** architecture consists of 2 GAT layers, each followed by non-linear ReLU activation function. Additionally, the output from GAT layers are then processed in Alpha and Beta Networks, which are integral components of the policy network, each consisting of a Linear layer to transform the input dimensions to target dimension and produces α and β variables that are required for sampling action from Beta distribution.

In contrast, **value network** has the same structure with policy network but omits the Alpha and Beta networks. Instead, it has a linear layer which transforms the output of the second GAT layer to a single scalar value.

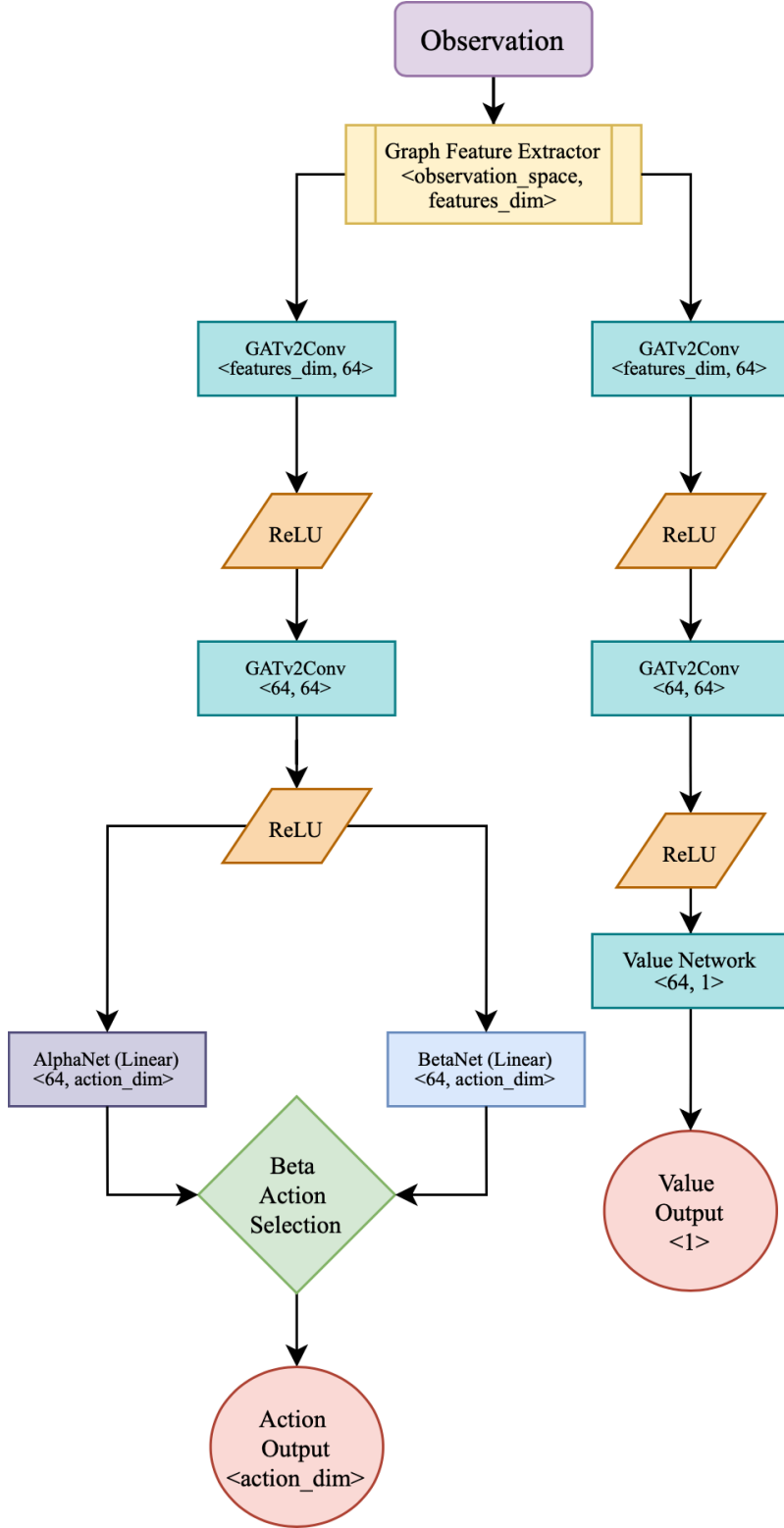


Figure 3.1: Network architecture of SatNet, illustrating the flow of data through the policy and value networks. The diagram highlights the use of GAT layers for message aggregation and the integration of Beta distribution for action selection within the policy network. The Alpha and Beta networks, essential for generating the parameters of the Beta distribution, are also depicted, showcasing the structure and functionality of SatNet in controlling articulated robots.

Chapter 4

Methodology

4.1 Introduction

The methodology used to construct SatNet leverages the skeletal structure of robotic systems, representing them as graphs where nodes correspond to joints, and edges denote physical connections. By utilizing GAT, the model effectively captures the relationships and dependencies between different parts of the robot., allowing for more precise decision-making. Beta distribution is used in action selection as it omits unintended bias [10]. The detailed methodology can be found at Section 4.2.

In order to capture a better understanding of how these methodologies align together to form SatNet, consider the process when the agent receives an observation vector s_τ . Initially, this vector is processed to form a graph G . Then, GAT layers transform this input through attention mechanism to produce a 64-dimensional representation. In the policy network, the alpha and beta networks compute the parameters required for beta sampling by transforming this 64-dimensional vector to action space dimension, resulting in an action sampled from this distribution. Meanwhile, the value network estimates the expected return from the final hidden states from GAT layers.

4.2 Model Methodology

4.2.1 Graph Construction

In reinforcement learning, at each time step τ the agent receives state observations s_τ which can also be called as features. For articulated robot continuous control tasks, these observations typically include the x, y, and z positions and velocities of the body parts, as well as the angles and angular velocities of the hinges. In order to process this observation data in the Graph Attention Network, these features have to be structured as a graph. By taking inspiration from the previous work [8] and structuring the agent as a graph $G = (V, E)$ where nodes $v \in V$ denote robot's joints and edges symbolize a physical connection between joints v_i and v_j . A feature vector is

attached to every node, containing the state information of that particular node v 's such as its x , y , z position and velocity, or angle and angular velocity as written in Algorithm 1. Whereas the paper [36] proposes a method to structure the robot as a graph automatically, in this research this operation is done manually for that specific agent before starting the RL process.

Algorithm 1 GraphFeatureExtractor

- 1: Define node_indices for each node (e.g., torso, limbs, etc.)
 - 2: Initialize an empty list node_features
 - 3: **for** each node_key in node_indices **do**
 - 4: Extract features from observation using node_indices[key]
 - 5: Pad features to match the features dimension
 - 6: Append padded_features to node_features list
 - 7: **end for**
 - 8: Stack node_features along a new dimension to form graph_data
 - 9: **return** graph_data
-

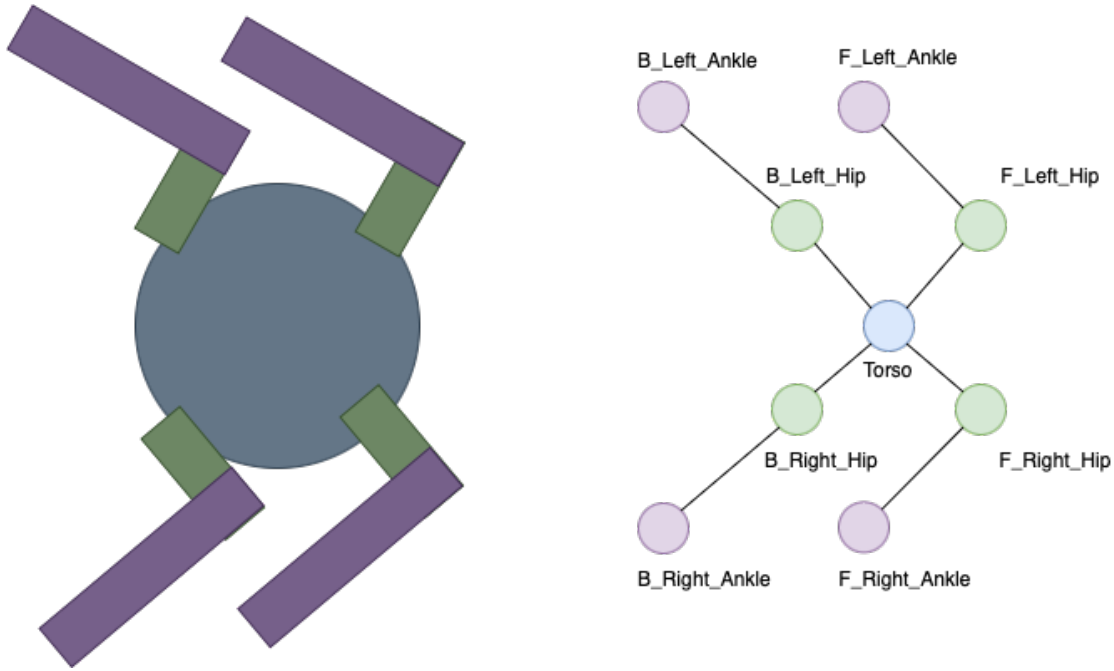


Figure 4.1: The graph structure of the Ant agent. The diagram illustrates the connection of nodes representing the robot's joints, where the torso serves as the root node, and each hip is connected to its corresponding ankle.

Consider the Ant illustrated in Figure 4.1. In this example, the body, which serves as the root node of this agent, is connected to each hip as shown. Each hip is then connected to its respective ankle, forming the graph structure of the ant environment in MuJoCo.

4.2.2 Policy-Value Network Architecture

SatNet uses the same network architecture for both policy and value networks. The architecture is designed to capture the relationships between the robot’s joints and their dynamics.

The core component of the network is the GAT layer, followed by non-linear ReLU activation function. Each GAT layer computes attention coefficients for each node and its neighbors, allowing the network to focus on the most relevant features of the input data.

Policy network is designed to compute the parameters for the Beta distribution, where the action will be sampled, using the on the aggregated features from the GAT layers. This design ensures that the actions are bounded within a specific range suitable for the physical constraints of the robot. The network consists of 2 GAT layers that aggregate and transform the input features, focusing on the most relevant aspects of the articulated robot’s current state through the attention mechanism. The output of the policy network includes the parameters α and β of the Beta Distribution, which are then used to sample actions.

The policy architecture consists of two **GATv2Conv** Layers each followed by ReLU activation functions.

- The first **GATv2Conv** layer transforms the input features (with the dimension of the node with the largest feature set) into a 64-dimensional space, followed by a ReLU activation function to introduce non-linearity.
- A second **GATv2Conv** layer further processes the features within the 64-dimensional space. Followed by another ReLU activation function.

Alpha & Beta Networks are linear layers that compute the parameters α and β for selecting an action from the Beta distribution. Both of these linear layers maps the 64-dimensional output of the GAT layers to the action space dimension of the agent.

Value network estimates the expected return from a given state, providing a measure of the long-term benefit of being in that state. In consists of two **GATv2Conv** layers identical to the

ones in the policy network, and a linear layer which maps the 64-dimensional output of the GAT layers into a single scalar value representing the estimated return.

4.2.3 Input Model

For each time step τ in the environment, the agent receives an observation vector s_τ , that encapsulates the current state of the environment. This observation vector is then processed using the Algorithm 1 which assign corresponding features to mapped nodes in the graph representation of the agent. Since different nodes may have varying numbers of features due to the heterogeneity in the state information, the algorithm addresses this by identifying the node with the maximum feature set and padding the feature vectors of all other nodes to match the size of this largest feature set. This padding process is crucial for maintaining a uniform input dimension for the Graph Attention Network (GAT) layers.

4.2.4 Propagation Model

The propagation model is the key component of the GAT layers, responsible for updating the hidden states of the nodes based on the attention coefficients and the features of neighboring nodes [14].

Calculation of Attention Coefficients

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(a^T [Wh_i \| Wh_j]))}{\sum_{k \in N_i} \exp(\text{LeakyReLU}(a^T [Wh_i \| Wh_k]))} \quad (4.1)$$

In GATs, message passing process involves calculating attention coefficients α_{ij} with the Equation 4.1, which indicates the importance of node j to node i . This process can be divided into two parts:

$$e_{ij} = \text{LeakyReLU}(a^T [Wh_i \| Wh_j]) \quad (4.2)$$

The first part begins with a linear transformation of the feature vectors of nodes i and j using a weight matrix W . These transformed features are then concatenated and passed through

a shared attention mechanism parameter vector a to obtain raw attention scores, using the Eq 4.2.

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in N_i} \exp(e_{ik})} \quad (4.3)$$

After obtaining the attention scores, they are normalized using the softmax function to produce the attention coefficients α_{ij} with Equation 4.3. These attention coefficients allow the network to focus on the most relevant neighboring nodes features. For example, in the Ant agent depicted in 4.1, the F_Right_Hip node sends weighted messages to its connected nodes, such as the torso and F_Right_Ankle.

Weighted Feature Aggregation

$$h'_i = ReLU \left(\sum_{j \in N_i} \alpha_{ij} W h_j \right) \quad (4.4)$$

The feature aggregation process involves computing a weighted sum of the features of neighboring nodes, enabling the network to capture the local structure and dependencies within the graph. For instance the updated feature vector of node i can be calculated using Equation 4.4 where h'_i is the updated feature vector of node i and N_i is the set of neighbors of node i .

4.2.5 Output Model

The output model generates actions and value estimates based on the final hidden states of the nodes after multiple GAT layers have been applied.

Action Selection The policy network computes the parameters of the Beta distribution from the aggregated features of the nodes. A sample from this distribution is selected as action a_τ to take for state s_τ . This distribution is more suitable for continuous control tasks rather than the Gaussian distribution as it introduces an unintended bias [10].

$$\pi_\theta(a_\tau | s_\tau) = \text{Beta}(\alpha_\theta(s_\tau), \beta_\theta(s_\tau)) \quad (4.5)$$

The Beta distribution is defined by its shape parameters α and β . The action a_τ is then sampled from the Beta distribution Equation 4.5 where $\alpha_\theta(s_\tau)$ and $\beta_\theta(s_\tau)$ are the parameters of the Beta distribution, which are functions of the state s_t and network parameters θ .

Value Estimation

$$V_\theta(s_\tau) = f(\text{GAT}(H)) \quad (4.6)$$

The value network uses the final hidden states of the nodes to estimate the value function, providing an estimate of the expected return from a given state. The output of the value network is a scalar value representing the estimated return calculated with Equation 4.6 where the GAT layer aggregates and processes the node features H to produce a meaningful representation of the state. This representation is then passed through a function f , which is a linear layer to map the high-dimensional aggregated features to a scalar value representing the estimated return. This value function helps in evaluating the long-term benefit of being in a particular state, guiding the policy network’s learning process.

4.2.6 Learning Algorithm

In RL, there are various algorithms to optimize the expected cumulative reward. This research uses PPO to optimize the objective function 3.1.

PPO is a policy gradient method that samples data through interaction with the environment and optimizes a surrogate objective function using stochastic gradient ascent. PPO uses a clipped objective function 4.7 to restrict the magnitude of policy updates, ensuring more stable and reliable learning. This objective function is designed to maximize the expected cumulative reward while maintaining the stability of policy updates [19].

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (4.7)$$

where $r_t(\theta) = \frac{\pi_\theta(a_\tau|s_\tau)}{\pi_{\theta_{\text{old}}}(a_\tau|s_\tau)}$ is the probability ratio of the new and old policies, $\hat{A}_t$ is the advantage estimate derived from [35], and ϵ is a hyperparameter controlling the clipping range.

Chapter 5

Experiments

5.1 Introduction

This chapter presents the experimental setup, results, and inferences drawn from the experiments conducted. The aim of these experiments is to evaluate the sample-efficiency, robustness and overall effectiveness of SatNet which serves as a policy network for articulated robots in continuous control tasks. Section 5.2 describes the experimental setup, Section 5.3 presents the results, and Section 5.4 discusses the inferences and insights gained.

5.2 Experimental Setup and Design

5.2.1 Experimental Setup and Computational Environment

This research uses Gymnasium [15] environments which are powered by MuJoCo [16] to conduct experiments. The data required for these experiments was generated within those simulated environments, meaning that there is no use of any external dataset. The experiments were conducted on a MacBook Pro featuring a 2.6 GHz 6-Core Intel Core i7 processor. The only programming language used in this paper is Python, version 3.12.4, with the primary libraries being Stable Baselines3 and PyTorch Geometric, which facilitated the implementation. The implementation code of this study is available in Appendix A1, while the raw data from model training, including metrics such as mean reward r_τ at time step τ , is provided in Appendix A3.

5.2.2 Experimental Design

`Ant-v4`, `Humanoid-v4`, `HumanoidStandup-v4`, and `HalfCheetah-v4` were chosen to evaluate the performance of SatNet as those environments encapsulates complex articulated robot structures. To isolate and assess the impact of the Beta distribution within the GAT framework, an additional policy was implemented and named as SatNet-Gaussian. This additional policy uses only the GAT layers and excludes the Beta distribution for action selection. The original SatNet model is called as SatNet-Beta, and the one that excludes Beta sampling and uses Gaussian

sampling, which is more conventional, for action selection is called SatNet-Gaussian for easier recognition. Both of these models were benchmarked with each other and the standard MLP policy to evaluate their overall effectiveness, sample efficiency and robustness. The experimental setup involves two distinct comparison experiments involving these three models.

The first experiment, conducted in subsection 5.3.1, compares the performance of the models while both training and testing, focusing on their mean rewards to assess sample-efficiency and overall effectiveness. The second experiment, conducted in subsection 5.3.2, simulates a scenario where the sensors of articulated robots got jammed, introducing noise with a ratio of 50% which affects only the angle measurements. This experiment is designed to evaluate the robustness of each model under interference.

5.2.3 Constraints, Assumptions, and Model Simplifications

Hardware Constraints Due to the absence of CUDA support on MacBooks with Intel processors, all computations were performed on the CPU. Consequently, the time required for training the models was significantly extended, which constrained the research’s ability to generate larger and diverse datasets.

Model Naming Convention The original SatNet model, which utilizes Beta sampling for action selection, is referred to as SatNet-Beta. For easier recognition, the version of the model that excludes Beta sampling and instead uses Gaussian sampling for action selection is named SatNet-Gaussian.

Model Simplifications The graph modelling is simplified as it gets complicated when agent has vast amount of joints. Take Humanoid-v4 environment as an example, this robot has 3 joints in it’s torso which are "torso", "abdomen", and "pelvis". To make the representation simpler, those 3 joints are merged into one node as detailed in A2.

Action Space Adjustments In continuous control tasks, the actions spaces are usually bounded around 0 with a standard deviation of 1. Given the bounded nature of Beta distribution outputs (between 0 and 1), the action spaces in environments were rescaled accordingly while using SatNet-Beta as it samples data from Beta distribution.

5.3 Results

In gymnasium environments, each agent has their own reward generating mechanism to compute average reward per each timestep. One can go to the documentation to see the reward mechanism of each agent [37]. During model training, selecting appropriate hyperparameters is crucial as they significantly impact the learning process. In order to identify the optimal hyperparameters for each model, a grid search was conducted. The details of the grid-search can be found in section A2. The following subsections present the results of the experiments.

5.3.1 Performance Benchmark

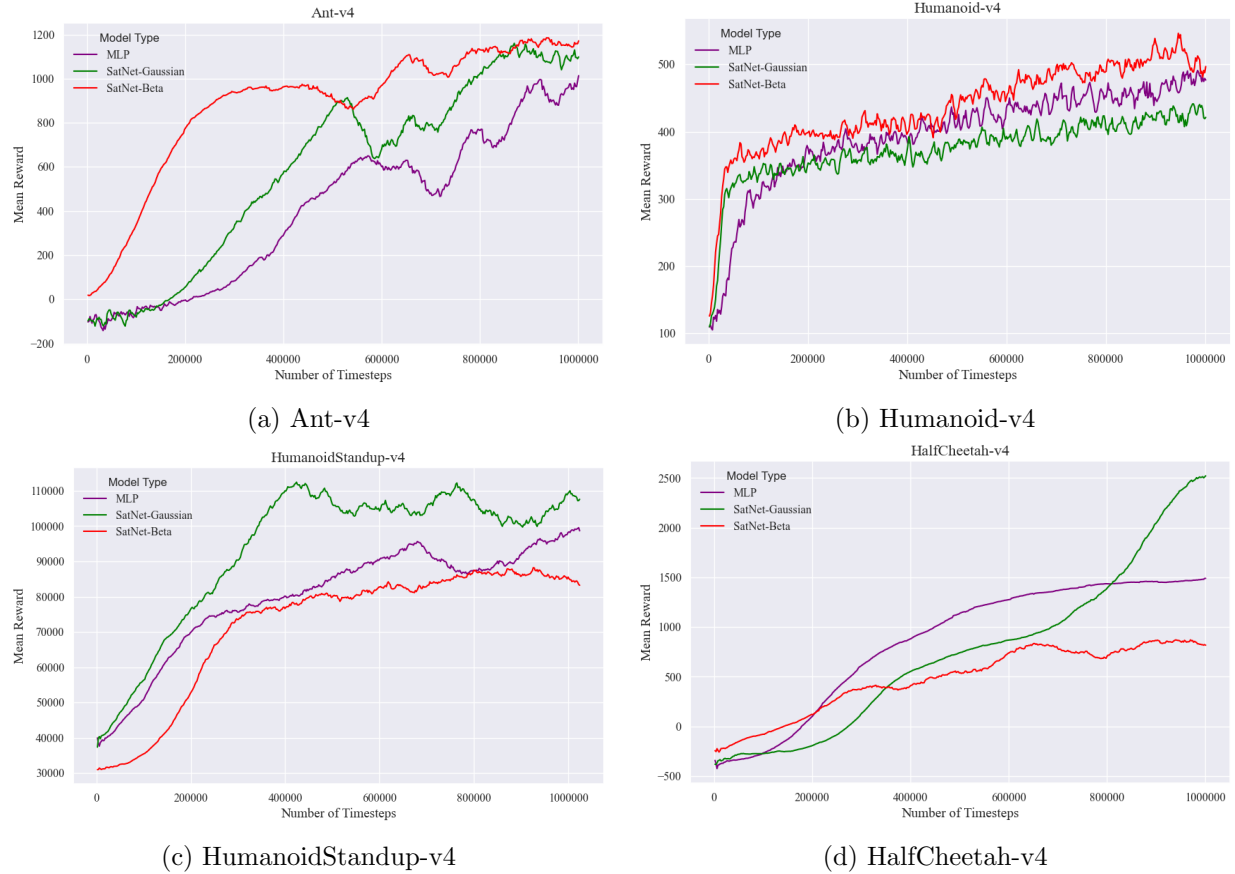


Figure 5.1: Training performance of SatNet-Beta, SatNet-Gaussian, and MLP over 1 million timesteps in the Ant-v4, HalfCheetah-v4, Humanoid-v4, and HumanoidStandup-v4 environments.

The Figure 5.1 illustrates the average reward returns over timesteps while training. This comparison provides insight into how each policy performs across various environments, highlighting differences in learning efficiency and overall effectiveness.

SatNet-Beta demonstrated superior performance in both **Ant-v4** and **Humanoid-v4** environments. In the **Ant-v4** environment, SatNet-Beta accumulated a mean reward about 1150 at the end of the training process, outperforming the SatNet-Gaussian and MLP models, which received mean rewards around 1100 and 1000, respectively. Notably, SatNet-Beta reached a total average reward around 1000 by 300,000 timesteps while MLP reached it after 1,000,000 timesteps, showcasing the superior sample efficiency of SatNet-Beta. This translates to a 70% reduction in the time required to reach peak performance compared to the MLP. In the **Humanoid-v4** environment, SatNet-Beta demonstrated higher performance than other policies by receiving the highest reward through the training process.

In contrast, the **SatNet-Gaussian** excelled in the **HumanoidStandup-v4** and **HalfCheetah-v4** environments. In the **HumanoidStandup-v4** environment, the SatNet-Gaussian model achieved a remarkable performance by reaching a total mean reward of 120,000, the highest reward observed across all environments in the chart. The high sample efficiency that GAT brings is evidenced by the model as it reaches this peak reward within just 400,000 timesteps. In contrast, the SatNet-Beta and MLP models managed to reach only around 75,000 in the same timeframe, highlighting the exceptional effectiveness and efficiency of the SatNet-Gaussian model in this particular environment. At the end of 1 million timesteps, SatNet-Gaussian accumulated the highest reward followed by MLP and SatNet-Beta. In **HalfCheetah-v4**, SatNet-Gaussian showcased perfect performance. MLP was leading the highest reward ranking until 800,000 timesteps, SatNet-Gaussian excelled after that timestep and gained over 1000 rewards in 200,000 timesteps, it ended up with a mean reward of approximately 2500 which is 67% higher than the MLP and a 300% higher reward than the SatNet-Beta model.

MLP policy, while not the top performer, displayed consistent and reasonable performance across all environments, indicating that it serves as a reliable baseline.

Table 5.1: Comparison of mean reward and standard deviation during testing for SatNet-Beta, SatNet-Gaussian, and MLP across different environments.

	Ant-v4		Humanoid-v4		HumanoidStandup-v4		HalfCheetah-v4	
	Mean Reward	Std	Reward Avg	Std	Reward Avg	Std	Reward Avg	Std
SatNet-Beta	1322.9	337.5	523.5	115.6	87249.4	12372.6	800.4	475.8
SatNet-Gaussian	1108.6	456.5	401.6	83.9	116458.2	16309.1	2602.0	411.1
MLP	1042.78	560.1	466.8	102.3	95923.6	12599.7	1467.0	124.9

The test performance comparison in Table 5.1 presents the metrics of the trained models when deployed in a test environment.

SatNet-Beta leads in the **Ant-v4** and **Humanoid-v4** environments, achieving the highest mean rewards of 1322.9 and 523.5, respectively, though with moderate standard deviations, indicating consistent performance.

In contrast, **SatNet-Gaussian** excels in the **HumanoidStandup-v4** and **HalfCheetah-v4** environments, achieving remarkable mean rewards of 116458.2 and 2602.0, respectively, despite having higher standard deviations, suggesting more variability in performance.

The **MLP** model, while generally consistent, tends to lag behind at least one of the specialized SatNet models.

SatNet-Beta has proven to be particularly sample-efficient and achieves higher rewards in fewer timesteps in environments where the agent’s dynamics can be modeled symmetrically and the actions taken are symmetrical, such as in **Humanoid-v4** and **Ant-v4**. However, SatNet-Gaussian shows better performance and sample efficiency in environments that require more fluid and dynamic actions, like **HalfCheetah-v4** and **HumanoidStandup-v4**. These results align with the research hypothesis that GAT layers can enhance sample efficiency when paired with the tailored action selection mechanism for the specific environment.

5.3.2 Robustness of Policies

The noise introduced in this experiment is Gaussian noise with a standard deviation equal to 50% of the sensor reading. For instance, if the sensor reads 10, the standard deviation of the noise would be 5, meaning the sensor could read a value typically within the range of approximately 5 to 15, depending on the random noise added. This noise is set to only affect the angle measurements. Each policy is evaluated over 50 episodes, and the average results are considered for the comparison.

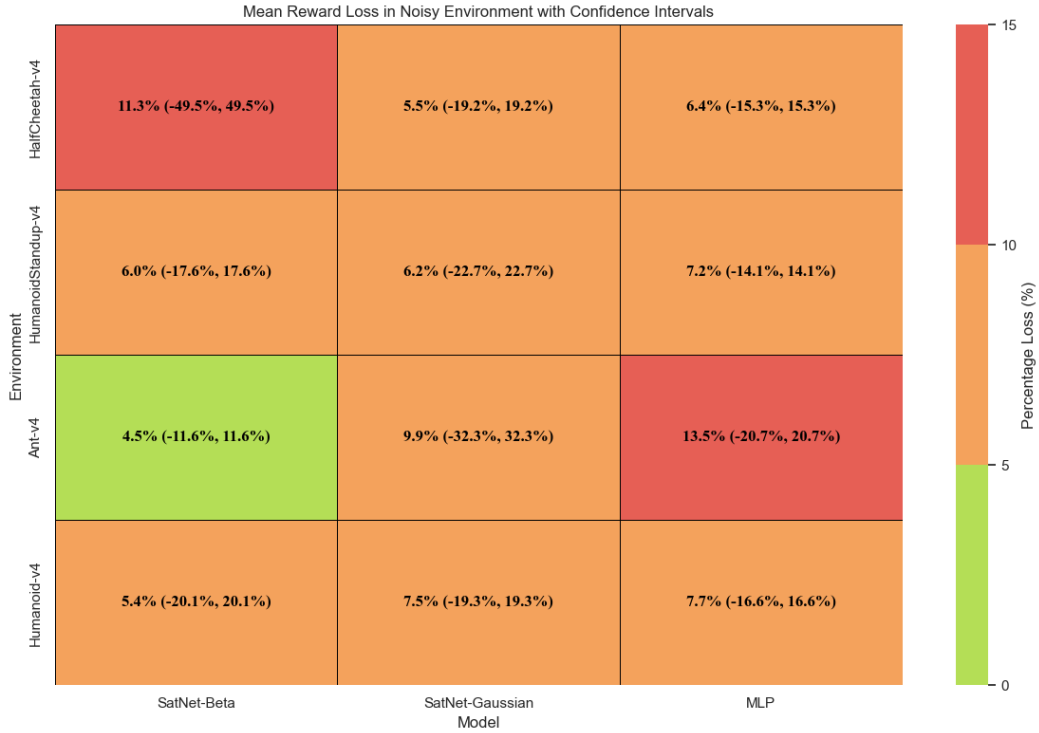


Figure 5.2: Heatmap illustrating the mean reward loss due to noise across different environments for SatNet-Beta, SatNet-Gaussian, and MLP.

From the heatmap depicted in Figure 5.2, it is evident that the SatNet-Beta model demonstrates the least performance loss across most of the environments, indicating its robustness to noise. However, it has the largest performance loss on **HalfCheetah-v4** environment which might be due to non-convergence of the model in that environment. The SatNet-Gaussian model also exhibits reliable robustness but not as promising as the SatNet-Beta model. On the other hand, the MLP model shows a weak robustness, having a significant decline in the Ant-v4 environment.

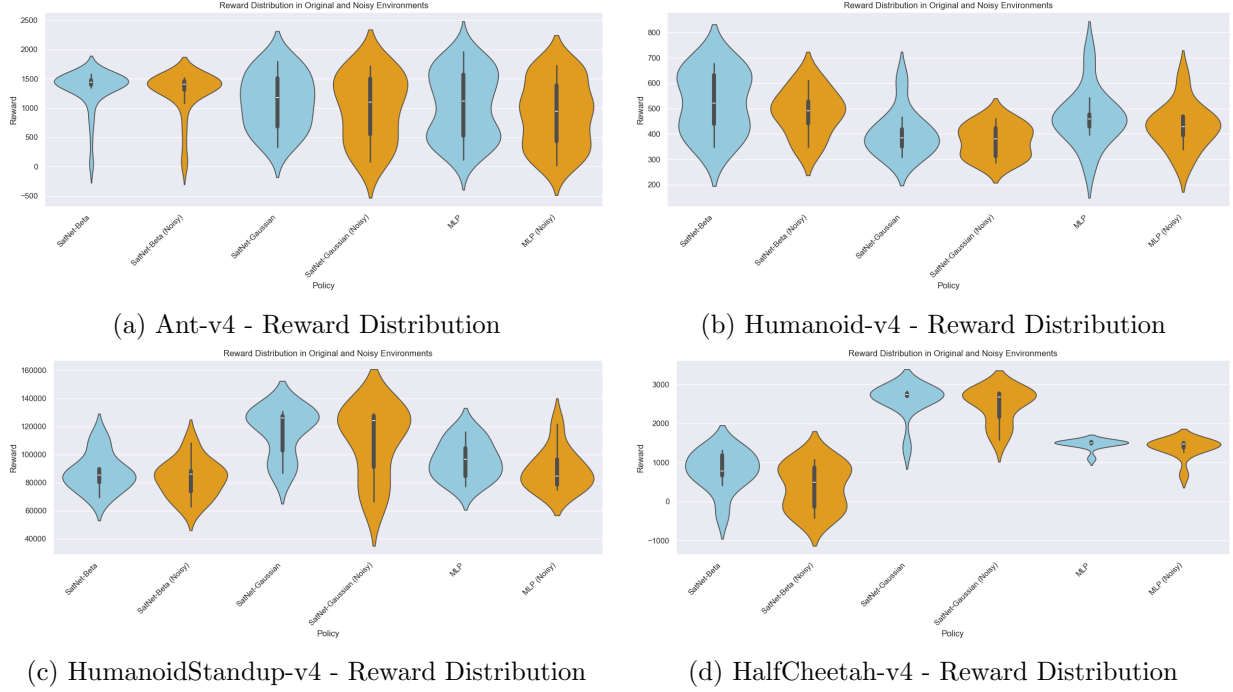


Figure 5.3: Violin plots showing the reward distribution of SatNet-Beta, SatNet-Gaussian, and MLP in original and noisy environments for Ant-v4, Humanoid-v4, HumanoidStandup-v4, and HalfCheetah-v4.

The violin plots provide a detailed view of the reward distribution and the effect of noise on the different models as depicted in Figure 5.3. The **SatNet-Beta** model consistently demonstrates superior robustness to noise, maintaining stable reward distributions with minimal performance degradation across all environments except **HalfCheetah-v4** which might be issued by the non-convergence of model. It performs particularly well in the **Ant-v4** and **HumanoidStandup-v4** environments, showing only slight reductions in rewards under noisy conditions. Conversely, the **SatNet-Gaussian** model, while achieving high rewards in noise-free settings, is sensitive to noise, resulting in slight performance declines and broader reward distributions. This pattern is evident in the **Ant-v4** and **Humanoid-v4** environments. The **MLP** model, although performing reasonably well in normal conditions, suffers the most under noise, with the highest performance loss across most of the environments, indicating it is the least robust model.

These robustness tests highlight the advanced noise resilience of the SatNet-Beta model across various environments, fulfilling this research’s aim to improve robustness with integrating GAT and Beta distribution to policy architecture.

5.4 Inferences

5.4.1 Discussion

The results of this research aligns with the hypothesis of this research which is integrating GATs into the agent’s policy architecture enhances both sample-efficiency, robustness and overall performance in continuous control tasks.

In environments where actions follow a symmetric pattern and precise control is vital such as Ant-v4 and Humanoid-v4, the combination of GATs with Beta distribution resulted in a superior performance as demonstrated by SatNet-Beta. SatNet-Beta tends to exploit the learned policy more effectively, leading to perform actions well-suited to these environment and led to better sample efficiency and overall results compared to other policies. Moreover, the action selection from Beta distribution is particularly effective in mitigating the bias caused by noisy environments. This robustness was evident in the robustness comparison subsection 5.3.2 where SatNet-Beta outperformed the other models. This exceptional performance makes SatNet-Beta particularly well-suited for applications in sectors like manufacturing and healthcare.

On the other hand, in environments that require more dynamic and exploratory actions such as HumanoidStandup-v4 and HalfCheetah-v4, the SatNet-Gaussian model outperformed other models. The flexibility provided by the Gaussian distribution appeared to be advantageous tasks where broader exploration of the action space was beneficial. However, this same flexibility made the SatNet-Gaussian model more sensitive to noise, as evidenced by its slightly reduced robustness compared to SatNet-Beta. From these insights, it can be claimed that SatNet-Gaussian is more appropriate for dynamic environments where rapid and adaptable responses are needed such as logistics.

Therefore, the integration of GATs enhances the learning capabilities and robustness of control policies. However, the full potential of this integration is realized only when the action selection mechanism is appropriately aligned with the environmental context and task requirements. Additionally, the use of Beta sampling within the SatNet Framework demonstrates superior noise resilience. These findings underscore the importance of a holistic approach to policy design, considering both architecture and action selection for optimal results.

Chapter 6

Professional and Ethical Issues

6.1 Introduction

The development of the Skeletal Attention Network for controlling articulated robots not only focuses on technological advancements but also considers broader impacts. Environmentally, the project emphasizes energy-efficient computational models and does not necessitate any need for physical prototypes by using simulation environments. Ethically, it promotes transparency in AI decision-making processes and avoids biases by generating its own data. Socially, the project aims to improve safety and accessibility in industries adopting robotic automation. Economically, it enhances cost-efficiency and reduces operational costs through improved sample efficiency and robustness, leading to potential savings in training time and resources. This comprehensive approach ensures that the SatNet project aligns with sustainable, ethical, and socially responsible engineering practices.

6.2 PESTEL Analysis

PESTEL analysis illustrated in Table 6.1, highlights that this research makes comprehensive approach to addressing political, economic, social, technological, environmental, and legal factors. By adhering to regulations, improving cost-efficiency, enhancing safety and accessibility, leveraging advanced technologies, focusing on sustainability, and ensuring compliance with legal standards, this work demonstrates a commitment to creating a responsible and beneficial solution for controlling articulated robots.

Table 6.1: PESTEL Analysis

Factor	Description
Political	This project adheres to the existing guidelines to ensure the safe and ethical deployment of AI in robotics. It also considers the potential impact of future regulations on the development and use of SatNet.
Economic	The SatNet project aims to enhance cost-efficiency and reduce operational costs for industries utilizing articulated robots. By improving sample efficiency and robustness, the project contributes to significant savings in training time and resources. This economic benefit can lead to increased use of robotic automation in various sectors.
Social	The social impact of this project includes improving safety and accessibility in industries that adopt robotic automation. However, it is essential to consider potential job displacement due to increased automation. Efforts should be made to retrain workers and integrate robots in ways that support human labor rather than replace it.
Technological	The integration of GATs and Beta distribution into the policy network of RL agents represents a significant technological advancement. This project leverages these cutting-edge technologies to improve the control of articulated robots, providing more precise and efficient solutions for complex tasks.
Environmental	Environmental considerations include the energy efficiency of the computational models used in the project. By focusing on sustainable practices in the design and implementation of SatNet, this project aims to minimize its environmental footprint. The use of simulation environments also reduces the need for physical prototypes, saving resources and energy.
Legal	Legal considerations involve adherence to safety standards and regulations governing the use of robotics and AI. This project ensures compliance with these legal requirements, promoting the responsible and lawful development of robotic technologies.

6.3 Ethical Considerations

In this study, ethical considerations were considered, and potential ethical violations were avoided through the use of appropriate procedures. The generation of data within simulated environments ensures that no personal information is processed or stored, protecting individual privacy. In addition, this paper emphasizes transparency in AI decision-making processes to avoid biases and discrimination. By promoting ethical AI, this research aligns with the broader goal of developing trustworthy and responsible technologies.

6.4 Inclusive Engineering Outcomes

This research equally considers the perspectives of all stakeholders. It proactively addresses biases and ensures that its benefits are accessible to the universal public. This work also anticipates future needs and trends in robotics and AI, ensuring that the solutions remain relevant and adaptable. Accessibility is a key focus; the code required for this research is provided in Section A1. Furthermore, the project aligns with the United Nations Sustainable Development Goals (SDGs), particularly SDG 9: Industry, Innovation, and Infrastructure, and SDG 12: Responsible Consumption and Production [38].

6.5 Conclusion

This chapter has revealed the professional and ethical responsibilities of engineers in the context of the SatNet project. By expanding the traditional PESTEL analysis to include ethical and inclusive approaches, this paper demonstrates a commitment to responsible engineering practices. The emphasis on sustainability, innovation, and accessibility complies with the United Nations Sustainable Development Goals, ensuring that the improvements in robotic control systems benefit society as a whole. Through adherence to ethical standards and industry regulations, the SatNet project sets a precedent for future research and development in the field of robotics.

Chapter 7

Conclusion

This research addressed the problem of improving the autonomy and efficiency of articulated robots by integrating GATs and Beta distribution into the policy architecture, hypothesizing that this approach would enhance sample efficiency, precision, and robustness. The problem was modeled using a Skeletal Attention Network (SatNet), which effectively captures the morphological relationships between robotic joints through a graph-based structure and integrating this data into the policy network. To verify the hypotheses, PPO was employed, with experiments conducted in simulated environments to compare the performance of SatNet against standard MLP policy. The results demonstrated that SatNet, particularly when using the Beta distribution for action selection, achieved higher performance in terms of sample efficiency and robustness, highlighting its potential for real-world applications. The findings suggest that the proposed integration could significantly advance the state of robotic control systems by enabling more efficient and reliable operations.

Despite the promising results demonstrated by the SatNet model, this study faced certain limitations, especially due to its reliance on simulated environments, which may not fully capture the complexities of real-world scenarios and could potentially limit the generalizability of the findings. Additionally, the computational intensity of the GAT-based models, suggests a need for more powerful hardware to achieve optimal results. Future work could involve extended hyperparameter tuning and benchmarking different reinforcement learning algorithms to further validate the efficiency and robustness of SatNet. Moreover, exploring the implementation of a dynamic action selection method that adapts to different control tasks could also enhance the model’s performance and adaptability.

References

- [1] David Silver, Aja Huang, Christopher J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–503, 2016.
- [2] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, Junhyuk Oh, Dan Horgan, Manuel Kroiss, Ivo Danihelka, Aja Huang, Laurent Sifre, Trevor Cai, John P Agapiou, Max Jaderberg, Alexander S Vezhnevets, Rémi Leblond, Tobias Pohlen, Valentin Dalibard, David Budden, Yury Sulsky, James Molloy, Tom L Paine, Caglar Gulcehre, Ziyu Wang, Tobias Pfaff, Yuhuai Wu, Roman Ring, Dani Yogatama, Dario Wünsch, Katrina McKinney, Oliver Smith, Tom Schaul, Timothy Lillicrap, Koray Kavukcuoglu, Demis Hassabis, Chris Apps, and David Silver. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.
- [3] Kenji Doya. Reinforcement Learning in Continuous Time and Space. *Neural Computation*, 12(1):219–245, 01 2000.
- [4] Marius-Constantin Popescu, Valentina Balas, Liliana Perescu-Popescu, and Nikos Mastorakis. Multilayer perceptron and neural networks. *WSEAS Transactions on Circuits and Systems*, 8, 07 2009.
- [5] Yan Duan, Xi Chen, Rein Houthooft, John Schulman, and Pieter Abbeel. Benchmarking deep reinforcement learning for continuous control. *CoRR*, abs/1604.06778, 2016.
- [6] Rongrong Liu, Florent Nageotte, Philippe Zanne, Michel de Mathelin, and Birgitta Dresp-Langley. Deep reinforcement learning for the control of robotic manipulation: A focussed mini-review. *CoRR*, abs/2102.04148, 2021.
- [7] M. Gori, G. Monfardini, and F. Scarselli. A new model for learning in graph domains. In *Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005.*, volume 2, pages 729–734 vol. 2, 2005.
- [8] Tingwu Wang, Renjie Liao, Jimmy Ba, and Sanja Fidler. Nervenet: Learning structured policy with graph neural networks. In *International conference on learning representations*, 2018.
- [9] Alberto Silvio Chiappa, Alessandro Marin Vargas, and Alexander Mathis. Dmap: a distributed morphological attention policy for learning to locomote with a changing body, 2022.
- [10] Po-Wei Chou, Daniel Maturana, and Sebastian Scherer. Improving stochastic policy gradients in continuous control with deep reinforcement learning using the beta distribution. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 834–843. PMLR, 06–11 Aug 2017.

- [11] J. Mukund Nilakantan, George Q. Huang, and S.G. Ponnambalam. An investigation on minimizing cycle time and total energy consumption in robotic assembly line systems. *Journal of Cleaner Production*, 90:311–325, 2015.
- [12] Junfan Lin, Zhongzhan Huang, Keze Wang, Xiaodan Liang, Weiwei Chen, and Liang Lin. Continuous transition: Improving sample efficiency for continuous control problems via mixup. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9490–9497, 2021.
- [13] Florian E. Dörner. Measuring progress in deep reinforcement learning sample efficiency, 2021.
- [14] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks, 2018.
- [15] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *CoRR*, abs/1606.01540, 2016.
- [16] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012.
- [17] Olivier Buffet, Olivier Pietquin, and Paul Weng. Reinforcement learning, 2020.
- [18] L. P. Kaelbling, M. L. Littman, and A. W. Moore. Reinforcement learning: A survey, 1996.
- [19] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.
- [20] Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Proceedings of the 12th International Conference on Neural Information Processing Systems, NIPS’99*, page 1057–1063, Cambridge, MA, USA, 1999. MIT Press.
- [21] Dong Han, Beni Mulyana, Vladimir Stankovic, and Samuel Cheng. A survey on deep reinforcement learning algorithms for robotic manipulation. *Sensors*, 23(7), 2023.
- [22] John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust region policy optimization, 2017.
- [23] Sham M Kakade. A natural policy gradient. In T. Dietterich, S. Becker, and Z. Ghahramani, editors, *Advances in Neural Information Processing Systems*, volume 14. MIT Press, 2001.
- [24] Jan Peters and Stefan Schaal. Reinforcement learning by reward-weighted regression for operational space control. In *Proceedings of the 24th International Conference on Machine Learning, ICML ’07*, page 745–750, New York, NY, USA, 2007. Association for Computing Machinery.
- [25] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning, 2019.
- [26] Yotam Hechtlinger, Purvasha Chakravarti, and Jining Qin. A generalization of convolutional neural networks to graph-structured data, 2017.

- [27] Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, and Maosong Sun. Graph neural networks: A review of methods and applications. *CoRR*, abs/1812.08434, 2018.
- [28] Fengyi Zhang, Zhiyong Liu, Fangzhou Xiong, Jianhua Su, and Hong Qiao. Wagnn: A weighted aggregation graph neural network for robot skill learning. *Robotics and Autonomous Systems*, 130:103555, 2020.
- [29] Fengyi Zhang, Fangzhou Xiong, and Zhiyong Liu. Learning individual features to decompose state space for robotic skill learning. In *2020 Chinese Control And Decision Conference (CCDC)*, pages 3169–3174, 2020.
- [30] Khadija Shaheen, Muhammad Abdullah Hanif, Osman Hasan, and Muhammad Shafique. Continual learning for real-world autonomous systems: Algorithms, challenges and frameworks. *Journal of Intelligent & Robotic Systems*, 105(1):9, 2022.
- [31] Aravind Rajeswaran, Kendall Lowrey, Emanuel Todorov, and Sham Kakade. Towards generalization and simplicity in continuous control, 2018.
- [32] Yixin Lin, Austin S. Wang, Eric Undersander, and Akshara Rai. Efficient and interpretable robot manipulation with graph neural networks, 2022.
- [33] Mert Arcan. Project proposal, 2024.
- [34] Martin L. Puterman. Chapter 8 markov decision processes. In *Stochastic Models*, volume 2 of *Handbooks in Operations Research and Management Science*, pages 331–434. Elsevier, 1990.
- [35] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation, 2018.
- [36] Tingwu Wang, Yuhao Zhou, Sanja Fidler, and Jimmy Ba. Neural graph evolution: Towards efficient automatic robot design. *CoRR*, abs/1906.05370, 2019.
- [37] Gymnasium. Gymnasium documentation. Accessed on 01 08, 2024.
- [38] United Nations. United nations: Sustainable development. Accessed on 02 08, 2024.

Appendix A: Raw Data

A1 Computer Code

```
from stable_baselines3.common.distributions import Distribution, sum_independent_dims
import torch as th
from torch import nn
from typing import Any, Dict, List, Optional, Tuple, TypeVar, Union, Callable
from torch.distributions import Beta
from torch_geometric.nn import GATv2Conv
from stable_baselines3.common.torch_layers import BaseFeaturesExtractor
import torch.nn.functional as F
from torch_geometric.data import Data
import gymnasium as gym
from gymnasium import spaces
from stable_baselines3.common.policies import ActorCriticPolicy
from stable_baselines3.common.type_aliases import Schedule
import numpy as np
from stable_baselines3.common.distributions import Distribution
from functools import partial
from stable_baselines3.common.preprocessing import get_action_dim

class GraphFeatureExtractor(BaseFeaturesExtractor):
    """
    A custom feature extractor for environments with graph-structured observations.

    Attributes:
        features_dim (int): The dimensionality of the features for each node in the graph.
        node_indices (dict): A dictionary mapping node names to their corresponding slices in the
        velocity_indices (dict): A dictionary mapping node names to their corresponding velocity slices in the
        edge_index (Tensor): A tensor representing the edges in the graph.
    """
    def __init__(self, observation_space: gym.Space, environment, features_dim = 11):
        """
        Initializes the GraphFeatureExtractor with observation space, environment, and feature dimension.

        Parameters:
            observation_space (gym.Space): The observation space from Gymnasium.
            environment (str): The environment name, which determines the node and edge structure.
            features_dim (int): The feature dimension for each node in the graph which is equal to the
            features_dim of the observation space.
        """
        super(GraphFeatureExtractor, self).__init__(observation_space, features_dim) # feature dimension

        if environment == "Humanoid-v4" or environment == "HumanoidStandup-v4":
            self.features_dim = 11
```

```

self.node_indices = {
    'torso': slice(0, 5),
    'abdomen': slice(5, 8),
    'right_hip': slice(8, 11),
    'right_knee': slice(11, 12),
    'left_hip': slice(12, 15),
    'left_knee': slice(15, 16),
    'right_shoulder': slice(16, 18),
    'right_elbow': slice(18, 19),
    'left_shoulder': slice(19, 21),
    'left_elbow': slice(21, 22),
}

self.velocity_indices = {
    'torso': slice(22, 28),          # 0
    'abdomen': slice(28, 31),        # 1
    'right_hip': slice(31, 34),       # 2
    'right_knee': slice(34, 35),      # 3
    'left_hip': slice(35, 38),        # 4
    'left_knee': slice(38, 39),       # 5
    'right_shoulder': slice(39, 41),  # 6
    'right_elbow': slice(41, 42),     # 7
    'left_shoulder': slice(42, 44),   # 8
    'left_elbow': slice(44, 45),      # 9
}

self.edge_index = th.tensor([
    [0, 1], [1, 0],
    [1, 2], [2, 1],
    [2, 3], [3, 2],
    [1, 4], [4, 1],
    [4, 5], [5, 4],
    [0, 6], [6, 0],
    [6, 7], [7, 6],
    [0, 8], [8, 0],
    [8, 9], [9, 8],
], dtype=th.long).t().contiguous()

if environment == "HalfCheetah-v4":
    self.features_dim = 5
    self.node_indices = {
        'f_tip': slice(0, 2),  # adjust these based on actual observation details
        'b_thigh': slice(2, 3),
        'b_shin': slice(3, 4),
        'b_foot': slice(4, 5),
        'f_thigh': slice(5, 6),
        'f_shin': slice(6, 7),
        'f_foot': slice(7, 8),
    }

```

```

self.velocity_indices = {
    'f_tip': slice(8, 11), # adjust these based on actual observation details
    'b_thigh': slice(11, 12),
    'b_shin': slice(12, 13),
    'b_foot': slice(13, 14),
    'f_thigh': slice(14, 15),
    'f_shin': slice(15, 16),
    'f_foot': slice(16, 17),
}

self.edge_index = th.tensor([
    [0, 4], [4, 0],
    [4, 1], [1, 4],
    [1, 2], [2, 1],
    [2, 3], [3, 2],
    [4, 5], [5, 4],
    [5, 6], [6, 5]
], dtype=th.long).t().contiguous()

if environment == "Ant-v4":
    self.features_dim = 11
    self.num_of_nodes = 9
    self.node_indices = {
        'torso': slice(0, 5),
        'hip_1': slice(5, 6), 'ankle_1': slice(6, 7),
        'hip_2': slice(7, 8), 'ankle_2': slice(8, 9),
        'hip_3': slice(9, 10), 'ankle_3': slice(10, 11),
        'hip_4': slice(11, 12), 'ankle_4': slice(12, 13),
    }
    self.velocity_indices = {
        'torso': slice(13, 19),
        'hip_1': slice(19, 20), 'ankle_1': slice(20, 21),
        'hip_2': slice(21, 22), 'ankle_2': slice(22, 23),
        'hip_3': slice(23, 24), 'ankle_3': slice(24, 25),
        'hip_4': slice(25, 26), 'ankle_4': slice(26, 27),
    }

# Predefine the edge_index based on the known structure of the Ant
self.edge_index = th.tensor([
    [0, 1], [1, 0],
    [0, 3], [3, 0],
    [0, 5], [5, 0],
    [0, 7], [7, 0],
    [1, 2], [2, 1],
    [3, 4], [4, 3],
    [5, 6], [6, 5],
    [7, 8], [8, 7]
])

```

```

        ], dtype=th.long).t().contiguous()

@property
def features_dim(self):
    """
    Retrieves the feature dimension.

    Returns:
    int: The current feature dimension.
    """

    return self._features_dim

@features_dim.setter
def features_dim(self, value):
    """
    Property to set the feature dimension.

    Parameters:
    value (int): The feature dimension to be set.
    """

    self._features_dim = value

def forward(self, observations: th.Tensor):
    """
    Forward pass through the feature extractor.

    Parameters:
    observations (Tensor): The observation tensor.

    Returns:
    Tensor: A tensor of node features for the graph.
    """

    if observations.dim() == 1:
        observations = observations.unsqueeze(0)

    node_features = []
    i = 0

    for key in self.node_indices.keys():
        # Extract features for all instances in the batch
        pos_features = observations[:, self.node_indices[key]]
        vel_features = observations[:, self.velocity_indices[key]]
        # Concatenate and pad features for each node across the batch

```

```

        combined_features = th.cat((pos_features, vel_features), dim=-1)
        padded_features = F.pad(combined_features, (0, self.features_dim - combined_features.shape[-1]))
        node_features.append(padded_features)

# Stack the node features to create a tensor with the shape required by the model
        node_features_tensor = th.stack(node_features, dim=1) # Adjust shape if necessary

        return node_features_tensor

class BetaDistribution(Distribution):
    """
    A custom distribution class that implements a Beta distribution for
    SatNet implementation as sb_3 does not have this distribution built-in.

    This class handles the creation of a Beta distribution, sampling from it, calculating log
    and other necessary operations for use in RL.
    """

    def __init__(self, action_dim: int):
        super().__init__()
        self.action_dim = action_dim
        self.alpha = None
        self.beta = None

    def proba_distribution_net(self, latent_dim: int) -> Tuple[nn.Module, nn.Module]:
        """
        Create the linear networks that produce the alpha and beta parameters of the Beta distribution.

        Parameters:
        latent_dim (int): The dimensionality of the latent features.

        Returns:
        Tuple[nn.Module, nn.Module]: The neural networks that produce alpha and beta parameters.
        """

        alpha_net = nn.Linear(latent_dim, self.action_dim)
        beta_net = nn.Linear(latent_dim, self.action_dim)
        return alpha_net, beta_net

    def proba_distribution(self, alpha: th.Tensor, beta: th.Tensor) -> 'BetaDistribution':
        """
        Create the Beta distribution given alpha and beta parameters.

        Parameters:
        alpha (Tensor): The alpha parameter tensor.
        beta (Tensor): The beta parameter tensor.

```

```

Returns:
BetaDistribution: The Beta distribution object.
"""

    self.distribution = Beta(alpha, beta)
    return self

def log_prob(self, actions: th.Tensor) -> th.Tensor:

    log_prob = self.distribution.log_prob(actions)
    return sum_independent_dims(log_prob)

def entropy(self) -> th.Tensor:
    entropy = self.distribution.entropy()
    return sum_independent_dims(entropy)

def sample(self) -> th.Tensor:
    actions = self.distribution.rsample()
    return actions

def mode(self) -> th.Tensor:
    actions = self.distribution.mean
    return actions

def actions_from_params(self, alpha: th.Tensor, beta: th.Tensor, deterministic: bool = False):
    self.proba_distribution(alpha, beta)
    return self.get_actions(deterministic=deterministic)

def log_prob_from_params(self, alpha: th.Tensor, beta: th.Tensor) -> Tuple[th.Tensor, th.Tensor]:
    actions = self.actions_from_params(alpha, beta)
    log_prob = self.log_prob(actions)
    return actions, log_prob

class GATv2ConvWrapper(nn.Module):
    """
    Wrapper class for GATv2Conv to enable its usage in nn.Sequential by internally managing a
    """

    def __init__(self, in_channels, out_channels):
        """
        Initialize the GATv2ConvWrapper.

        Parameters:
        in_channels (int): Number of input features per node.
        out_channels (int): Number of output features per node.
        """

        super(GATv2ConvWrapper, self).__init__()

```



```

self.gatv2conv = GATv2Conv(in_channels, out_channels)

# Define a static edge_index
# The edge_index should be manually defined for the specific environment.
# Uncomment the appropriate edge_index definition depending on the environment.

# Ant
self.edge_index = th.tensor([
    [0, 1], [1, 0],
    [0, 3], [3, 0],
    [0, 5], [5, 0],
    [0, 7], [7, 0],
    [1, 2], [2, 1],
    [3, 4], [4, 3],
    [5, 6], [6, 5],
    [7, 8], [8, 7]
], dtype=th.long).t().contiguous()

# Half-Cheetah
"""
self.edge_index = th.tensor([
    [0, 4], [4, 0],
    [4, 1], [1, 4],
    [1, 2], [2, 1],
    [2, 3], [3, 2],
    [4, 5], [5, 4],
    [5, 6], [6, 5]

    ], dtype=th.long).t().contiguous()
"""

# Humanoid
"""
self.edge_index = th.tensor([
    [0, 1], [1, 0],
    [1, 2], [2, 1],
    [2, 3], [3, 2],
    [1, 4], [4, 1],
    [4, 5], [5, 4],
    [0, 6], [6, 0],
    [6, 7], [7, 6],
    [0, 8], [8, 0],
    [8, 9], [9, 8],

    ], dtype=th.long).t().contiguous()
"""

```

```

def forward(self, x):
    """
    Forward pass through the GATv2Conv layer with a static edge_index.

    Parameters:
    x (Tensor): The input node features, which can be either a single graph or a batch of

    Returns:
    Tensor: The output node features after applying the GATv2Conv layer.
    """

    if x.dim() == 3: # Batch processing
        outputs = []
        # Process each graph in the batch independently
        for single_graph_features in x:
            output = self.gatv2conv(single_graph_features, self.edge_index)
            outputs.append(output)
        return th.stack(outputs, dim=0)
    # Process a single graph
    return self.gatv2conv(x, self.edge_index)

class SatNet(nn.Module):
    """
    SatNet is a neural network model that utilizes graph attention layers (GATv2Conv)
    for policy and value function approximation in reinforcement learning.

    This model is particularly designed for environments where observations are structured
    as graphs, and the relationships between different parts of the state space can be
    effectively captured by graph attention networks.
    """

    def __init__(
        self,
        feature_dim: int,
        last_layer_dim_pi: int = 64,
        last_layer_dim_vf: int = 64,
    ):
        """
        Initialize the SatNet model.

        Parameters:
        feature_dim (int): The dimensionality of the input features
        last_layer_dim_pi (int): The dimensionality of the last hidden layer in the policy net
        last_layer_dim_vf (int): The dimensionality of the last hidden layer in the value net
        """
        super().__init__()

```

```

self.latent_dim_pi = last_layer_dim_pi
self.latent_dim_vf = last_layer_dim_vf

# Policy network
self.policy_net = nn.Sequential(
    GATv2ConvWrapper(feature_dim, last_layer_dim_pi), nn.ReLU(),
    GATv2ConvWrapper(last_layer_dim_pi, last_layer_dim_pi), nn.ReLU(),
)
# Value network
self.value_net = nn.Sequential(
    GATv2ConvWrapper(feature_dim, last_layer_dim_vf), nn.ReLU(),
    GATv2ConvWrapper(last_layer_dim_vf, last_layer_dim_vf), nn.ReLU(),
)

def forward(self, features: th.Tensor) -> Tuple[th.Tensor, th.Tensor]:
    """
    Forward pass to compute both the policy (actor) and value (critic) outputs.

    Parameters:
    features (Tensor): The input features tensor.

    Returns:
    Tuple[Tensor, Tensor]: The policy output (actions) and the value output (state value).
    """

    return self.forward_actor(features), self.forward_critic(features)

def forward_actor(self, features: th.Tensor) -> th.Tensor:
    """
    Forward pass through the policy network to compute the actions.

    Parameters:
    features (Tensor): The input features tensor.

    Returns:
    Tensor: The actions computed by the policy network.
    """
    actions = self.policy_net(features)
    actions = actions.mean(dim=1) # Aggregate the actions across the node dimension
    return actions

def forward_critic(self, features: th.Tensor) -> th.Tensor:
    """
    Forward pass through the value network to compute the state value.

    Parameters:

```

```

        features (Tensor): The input features tensor.

        Returns:
        Tensor: The state value computed by the value network.
        """
        value = self.value_net(features)
        value = value.mean(dim=1) # Aggregate the actions across the node dimension
        return value

class SatNetBeta(ActorCriticPolicy):
    """
    SatNetBeta is an actor-critic policy that uses SatNet for feature extraction
    and a Beta distribution in order to sample actions from the Beta distribution.
    This class integrates the custom SatNet architecture with the Beta distribution
    to allow for precise action spaces.
    """
    def __init__(
        self,
        observation_space: spaces.Space,
        action_space: spaces.Space,
        lr_schedule: Callable[[float], float],
        last_layer_dim_pi: int = 64,
        last_layer_dim_vf: int = 64,
        *args,
        **kwargs,
    ):
        self.last_layer_dim_pi = last_layer_dim_pi
        self.last_layer_dim_vf = last_layer_dim_vf
        super().__init__(
            observation_space,
            action_space,
            lr_schedule,
            # Pass remaining arguments to base class
            *args,
            **kwargs,
        )

    def _build_mlp_extractor(self) -> None:
        """
        Build the MLP (Multi-Layer Perceptron) extractor using the SatNet architecture.
        This replaces the default MLP extractor in the base ActorCriticPolicy with a custom
        graph-based network.
        """

        self.mlp_extractor = SatNet(self.features_dim, last_layer_dim_pi=self.last_layer_dim_pi)

    def _build(self, lr_schedule: Schedule) -> None:

```

```

self.action_dist = BetaDistribution(get_action_dim(self.action_space))

self._build_mlp_extractor()
latent_dim_pi = self.mlp_extractor.latent_dim_pi

if isinstance(self.action_dist, BetaDistribution):
    self.alpha_net, self.beta_net = self.action_dist.proba_distribution_net(latent_dim_pi)
else:
    raise NotImplementedError(f"Unsupported distribution '{self.action_dist}'.")

self.value_net = nn.Linear(self.mlp_extractor.latent_dim_vf, 1)

if self.ortho_init:
    module_gains = {
        self.features_extractor: np.sqrt(2),
        self.mlp_extractor: np.sqrt(2),
        self.alpha_net: 0.01,
        self.beta_net: 0.01,
        self.value_net: 1,
    }
    if not self.share_features_extractor:
        del module_gains[self.features_extractor]
        module_gains[self.pi_features_extractor] = np.sqrt(2)
        module_gains[self.vf_features_extractor] = np.sqrt(2)

    for module, gain in module_gains.items():
        module.apply(partial(self.init_weights, gain=gain))

# Setup optimizer with initial learning rate
self.optimizer = self.optimizer_class(self.parameters(), lr=lr_schedule(1), **self.optimizer_kwargs)

def _get_action_dist_from_latent(self, latent_pi: th.Tensor) -> Distribution:
    """
    Compute the action distribution with Beta distribution.

    Parameters:
        latent_pi (Tensor): The latent representation from the policy network.

    Returns:
        Distribution: The probability distribution of actions.
    """
    if isinstance(self.action_dist, BetaDistribution):
        softplus = th.nn.Softplus() # Apply softplus to ensure positivity of alpha and beta
        alpha = softplus(self.alpha_net(latent_pi)) + 1.0

```

```

        beta = softplus(self.beta_net(latent_pi)) + 1.0
        return self.action_dist.proba_distribution(alpha, beta)
    else:
        return super()._get_action_dist_from_latent(latent_pi)

class SatNetGaussian(ActorCriticPolicy):
    """
    SatNetGaussian is an actor-critic policy that uses SatNet for feature extraction
    and a Gaussian distribution for modeling the policy. This class integrates the custom
    SatNet architecture with the Gaussian distribution to allow for continuous action spaces.
    """

    def __init__(
        self,
        observation_space: spaces.Space,
        action_space: spaces.Space,
        lr_schedule: Callable[[float], float],
        last_layer_dim_pi: int = 64,
        last_layer_dim_vf: int = 64,
        *args,
        **kwargs,
    ):
        self.last_layer_dim_pi = last_layer_dim_pi
        self.last_layer_dim_vf = last_layer_dim_vf
        super().__init__(
            observation_space,
            action_space,
            lr_schedule,
            # Pass remaining arguments to base class
            *args,
            **kwargs,
        )

    def _build_mlp_extractor(self) -> None:
        self.mlp_extractor = SatNet(self.features_dim, last_layer_dim_pi=self.last_layer_dim_pi)

```

EXAMPLE USAGE

```

from gymnasium.wrappers import RescaleAction
from stable_baselines3 import PPO
"""
Note that environment should be manually modelled and set in Gatu2ConvWrapper and Graph Feature
"""

env = gym.make("Ant-v4")
# SatNet-Beta
env = RescaleAction(env, min_action=0.0, max_action=1.0)
SatNet_Beta = PPO(
    SatNetBeta,
    env,
    policy_kwargs= dict(
        features_extractor_class=GraphFeatureExtractor,
        features_extractor_kwargs=dict(environment="Ant-v4")
    ),
    verbose=1,
    batch_size=128,
)

# SatNet-Gaussian
SatNet_Gaussian = PPO(
    SatNetGaussian,
    env,
    policy_kwargs= dict(
        features_extractor_class=GraphFeatureExtractor,
        features_extractor_kwargs=dict(environment="Ant-v4")
    ),
)

# MLP
MLP = PPO("MlpPolicy", env, verbose=1)

SatNet_Beta.learn(1000)
SatNet_Gaussian.learn(1000)
MLP.learn(1000)

```

A2 Details of the Experimental Settings

A2.1 Hyperparameter Selection

Tables below summarizes the different values tried for various hyperparameters, which were selected to identify the most effective combinations for training the SatNet and MLP models.

Table A.1: MLP: Hyperparameter Grid Search Settings

MLP	Value Tried
Network Shape	[64, 64], [128,128], [256, 256]
Required # Steps Before Update	2048, 4096
Discount Factor (γ)	0.98, 0.99, 0.995
GAE	0.9, 0.95, 0.99
Clip Range	0.1, 0.2, 0.3
Learning Rate	1e-4, 3e-4, 1e-3
Batch size for PPO	32, 64, 128

Table A.2: SatNet: Hyperparameter Grid Search Settings

SatNet	Value Tried
Network Shape	[64, 64], [128,128]
Required # Steps Before Update	2048, 4096
Clip Range	0.1, 0.2, 0.3
Learning Rate	1e-4, 3e-4, 1e-3
Batch size for PPO	32, 64, 128

The tables A.1 A.2 include hyperparameters such as network shape, the number of steps required before an update, discount factor (γ), Generalized Advantage Estimation (GAE) parameter, clip range, learning rate, batch size, and additional model-specific settings. To reduce the time required for grid search on SatNet, GAE and Discount factor are removed from the search, and the [256, 256] Network shape is also excluded since it increases the computation time. Following the grid search, the optimized hyperparameters are presented in Tables A.3 A.4.

Table A.3: SatNet: Chosen Hyperparameters

SatNet	Value
Network Shape	[64, 64]
Required # Steps Before Update	2048
Clip Range	0.2
Learning Rate	3e-4
Batch size for PPO	64

Table A.4: MLP: Chosen Hyperparameters

MLP	Value
Network Shape	[64, 64]
Required # Steps Before Update	2048
Discount Factor (γ)	0.99
GAE	0.95
Clip Range	0.2
Learning Rate	3e-4
Batch size for PPO	64

A2.2 Graph Simplification

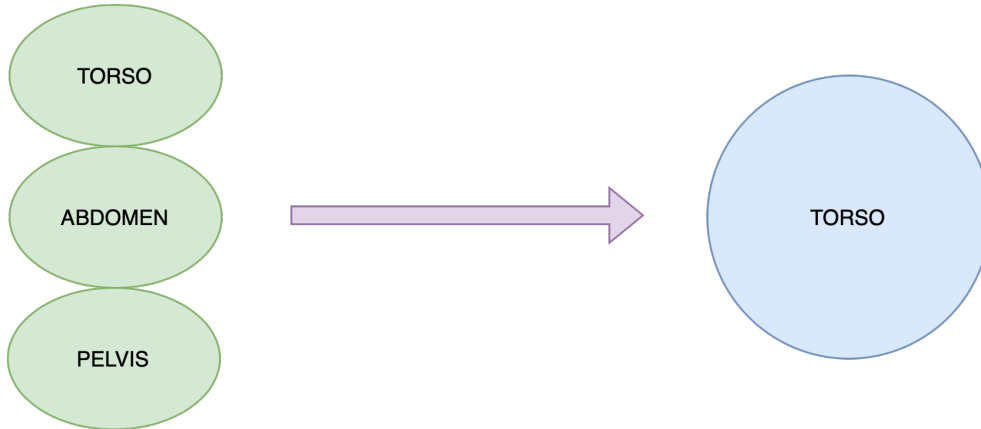


Figure A.1: Graph Simplification of the Humanoid Structure: The Torso, Abdomen, and Pelvis are Combined into a Single Root Node Labeled as Torso

This simplification is specifically applied to the Humanoid structure because the number of nodes directly influences the computation time. By reducing the number of nodes through combining the Torso, Abdomen, and Pelvis into a single root node, model becomes more computationally efficient without sacrificing the essential structure of the humanoid figure.

A3. Raw Data and Detailed Results

The data generated during this research is extensive, with each model being logged every 2,048 timesteps up to 1 million iterations. To streamline the presentation, only key values recorded at every 51,200 timesteps have been included for the MLP, SatNet-Gaussian, and SatNet-BETA models across each environment. The data generated during training is given in Tables A.5, A.6 and the evaluation results are given in Table A.7.

Table A.5: Mean Reward Achieved by MLP, SatNet-GAUSSIAN, and SatNet-BETA Models in Ant-v4 and Humanoid-v4 Environments at 51.200-Timestep Intervals

	Ant-v4			Humanoid-v4		
Timesteps	MLP	SatNet-Gaussian	SatNet-Beta	MLP	SatNet-Gaussian	SatNet-Beta
51200	-86.59	-76.04	125.56	235.18	317.31	359.19
102400	-33.49	-61.43	348.07	299.84	337.56	359.67
153600	-40.45	-22.44	600.65	336.86	336.49	399.65
204800	-9.04	68.13	789.99	373.97	351.51	392.68
256000	37.15	192.56	886.89	377.16	343.79	394.71
307200	96.45	352.49	937.96	371.86	355.74	407.07
358400	183.63	471.31	961.33	399.09	381.68	407.05
409600	322.12	589.08	962.38	394.10	356.45	425.90
460800	456.38	741.59	967.29	402.67	365.63	421.79
512000	543.15	892.58	897.23	413.04	380.60	452.35
563200	638.75	767.59	907.00	446.36	383.79	471.11
614400	597.10	701.98	1006.40	429.01	388.92	447.13
665600	583.77	832.13	1077.39	435.57	405.21	470.97
716800	486.80	779.67	1022.85	453.05	415.76	493.38
768000	708.40	959.44	1095.26	473.36	403.37	488.30
819200	719.28	1056.57	1134.50	449.86	400.43	501.16
870400	833.77	1160.38	1139.91	463.65	414.09	507.64
921600	995.03	1108.40	1155.56	477.35	410.11	521.66
972800	937.77	1084.32	1151.31	470.18	438.04	494.68
1024000	1013.53	1098.52	1171.39	476.86	421.31	496.99

Table A.6: Mean Reward Achieved by MLP, SatNet-GAUSSIAN, and SatNet-BETA Models in HumanoidStandup-v4 and HalfCheetah-v4 Environments at 51.200-Timestep Intervals

	HumanoidStandup-v4			HalfCheetah-v4		
Timesteps	MLP	SatNet-Gaussian	SatNet-Beta	MLP	SatNet-Gaussian	SatNet-Beta
51200	43 956.07	47 106.36	32 478.85	-333.52	-279.99	-150.23
102400	51 512.62	56 579.75	35 610.71	-264.09	-272.10	-79.39
153600	62 829.13	68 839.04	42 653.51	-115.97	-249.59	20.54
204800	70 637.79	76 748.91	53 912.69	129.26	-182.83	131.56
256000	74 719.73	83 879.08	66 723.10	406.99	-63.88	297.29
307200	76 049.33	92 348.35	74 347.02	635.67	161.48	386.13
358400	78 957.84	103 118.71	76 666.05	794.39	419.82	389.03
409600	80 123.42	111 004.52	77 836.75	908.48	573.81	429.07
460800	82 926.72	108 219.27	79 772.58	1029.65	671.49	506.80
512000	86 693.55	105 565.72	79 511.54	1152.12	760.54	557.32
563200	88 529.31	105 707.73	81 649.49	1235.09	827.52	602.16
614400	91 001.55	106 176.45	82 816.92	1300.59	878.13	762.12
665600	94 884.66	103 033.11	81 374.02	1345.57	950.54	827.10
716800	90 629.03	107 425.74	84 063.42	1380.00	1083.38	753.86
768000	87 152.39	111 053.59	85 826.72	1417.27	1274.23	722.37
819200	87 557.55	103 528.07	86 730.89	1439.75	1480.59	751.54
870400	89 175.36	101 974.67	87 871.88	1456.86	1815.28	825.60
921600	94 181.58	102 630.57	86 955.94	1449.87	2198.97	857.24
972800	96 411.12	104 743.12	85 314.99	1468.01	2478.42	862.05
1024000	98 673.55	107 602.52	83 231.12	1490.55	2522.73	817.73

Table A.7: Evaluation Results for Different Models and Environments: Maximum, Mean, and Standard Deviation of Rewards Across Original and Noisy Conditions

Environment	Model	Environment Condition	Max Reward	Mean Reward	Std Reward
HalfCheetah-v4	SatNet-Beta	Original	1317.82	800.44	475.78
HalfCheetah-v4	SatNet-BETA	Noisy	1083.17	709.59	538.03
HalfCheetah-v4	MLP	Original	1541.94	1467.00	124.98
HalfCheetah-v4	MLP	Noisy	1531.69	1372.76	243.55
HalfCheetah-v4	SatNet-GAUSSIAN	Original	2835.54	2602.06	411.18
HalfCheetah-v4	SatNet-GAUSSIAN	Noisy	2786.50	2458.53	425.32
HumanoidStandup-v4	SatNet-BETA	Original	112 568.06	87 249.43	12 372.63
HumanoidStandup-v4	SatNet-BETA	Noisy	108 211.25	82 038.10	10 971.57
HumanoidStandup-v4	MLP	Original	116 255.09	95 923.63	12 599.74
HumanoidStandup-v4	MLP	Noisy	121 620.87	89 031.88	13 762.95
HumanoidStandup-v4	SatNet-GAUSSIAN	Original	130 605.62	116 458.28	16 309.18
HumanoidStandup-v4	SatNet-GAUSSIAN	Noisy	129 201.64	109 261.26	23 641.63
Ant-v4	SatNet-BETA	Original	1578.54	1322.91	337.53
Ant-v4	SatNet-BETA	Noisy	1520.92	1263.35	377.26
Ant-v4	MLP	Original	1962.64	1042.78	560.14
Ant-v4	MLP	Noisy	1731.23	901.65	553.41
Ant-v4	SatNet-GAUSSIAN	Original	1797.40	1108.58	456.47
Ant-v4	SatNet-GAUSSIAN	Noisy	1721.40	998.47	547.37
Humanoid-v4	SatNet-BETA	Original	677.40	523.53	115.65
Humanoid-v4	SatNet-BETA	Noisy	610.73	495.46	84.52
Humanoid-v4	MLP	Original	707.40	466.74	102.32
Humanoid-v4	MLP	Noisy	616.49	431.00	84.50
Humanoid-v4	SatNet-GAUSSIAN	Original	611.78	401.62	83.89
Humanoid-v4	SatNet-GAUSSIAN	Noisy	460.41	371.50	59.80